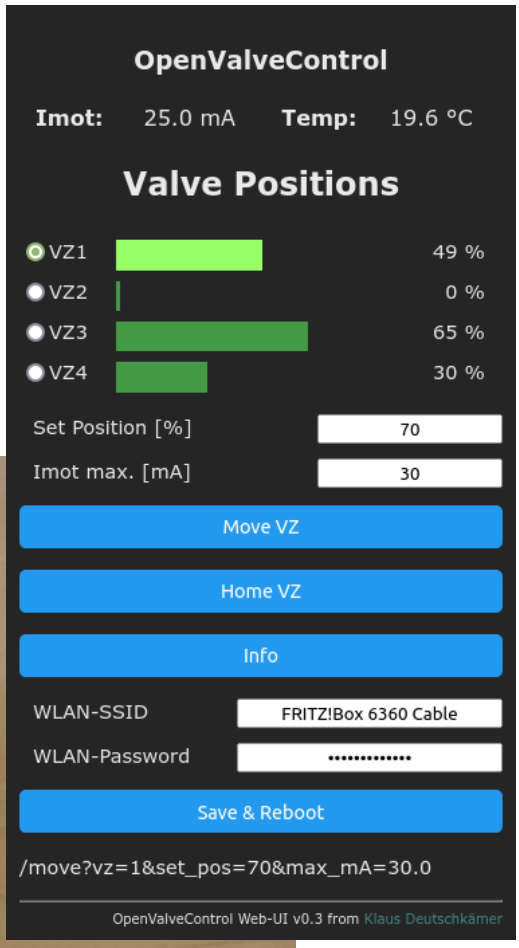



OpenValveControl
Imot: 25.0 mA Temp: 19.6 °C
Valve Positions
VZ1 49 %
VZ2 0 %
VZ3 65 %
VZ4 30 %
Set Position [%] 70
Imot max. [mA] 30
Move VZ
Home VZ
Info
WLAN-SSID FRITZ!Box 6360 Cable
WLAN-Password
Save & Reboot
/move?vz=1&set_pos=70&max_mA=30.0
OpenValveControl Web-UI v0.3 from Klaus Deutschkämer

Content

1 Introduction ... 3
1.1 Features ... 3
2 Initial Setup ... 4
2.1 Flashing the "ESP8266 D1-mini" for the first time ... 4
2.1.1 Arduino IDE (2.2.1) ... 4
2.1.2 tasmotizer ... 4
2.1.3 Command line using "esptool.py" ... 5
2.2 WiFi Configuration ... 5
2.3 Reboot and Login to your home network ... 7
2.4 Additional settings (ovc.ini) ... 8
2.5 Optional JavaScript file (customize.js) ... 9
2.6 First driving trials ... 11
2.6.1 Delivery status of the drives ... 11
2.6.2 Checking the reference run ... 11
3 Controlling with node-RED ... 13
3.1 Adjusting the valves using node-RED ... 13
3.1.1 Task definition ... 13
3.1.2 Specifying a setpoint ... 14
3.1.3 flow: "Subscribe" thermostat values from our MQTT broker ... 14
3.1.4 flow: Convert temperature difference into valve position ... 15
3.1.5 flow: Output the query parameters as http GET request ... 17
4 Driving the motors by PWM ... 19
4.1 Introduction to the H-bridge ... 19
4.2 PWM generation and interrupts ... 20
4.3 Acquisition of motor current ... 21
4.3.1 Exemplary measurement results ... 22
4.4 Measuring the motor inductance ... 23
4.5 Measuring the motor speed ... 25
4.6 Measuring the back EMF ... 26
4.6.1 Comparing Simulation and Measurement ... 28
4.6.2 Back EMF sample oscillograms ... 30
4.7 Calculating the motor constant k_e ... 31
5 Motor simulation with LTspice XVII ... 32
5.1 LTspice Circuit diagram ... 32
6 Data Logger ... 33
6.1 Measurements in the real field ... 34
7 Motor position control using BEMF? ... 35

1 Introduction

Open Valve Control is a low cost smart control for motorized 3 V floor heating valve actuators (like HmIP VdMOT and others) with web interface. The web interface (ESP-ValveControl) uses an ESP8266 D1-mini and the motor control (PIC-ValveControl) uses a PIC18F16Q41. All components are low cost and pluggable.

1.1 Features

- Connect up to 4 motorized, 3 V floor heating valve actuators (like HmIP VdMOT and similars)
- OLED Display (visualization of valve positions, motor current, temperature and status)
- WiFi-Interface (visualization, command execution and status requests using GET, POST, JSON)
- Supports DS18B20 (heating flow temperature)
- Filesystem support on ESP
- (upload web-UI "/index.html", setting data "/ovc.ini" and custom labels "/customize.js")
- Access point (until local WiFi has been configured)
- OTA-Support for ESP (Over-the-Air Update)
- OTA-Support (Over-the-Air Bootloader) for PIC-µC
- OTA-Write to PIC-EEPROM
- MQTT Support
- Data logger for motor current and speed (V_{BEMF})
- Examples for NodeRed, MQTT (, FHEM, Raspberry Pi, ...) - see Documentation.

While the source code and doxygen documentation is available on github, this documentation gives some additional insights in the project, especially ist gives some hints for installation and setup. But also the basics of running a motor using PWM and measuring current and back EMF voltage is provided.

This document will be continued as time permits. Feel free to contact me, if you miss certain topics. Thank You.

2 Initial Setup

Prerequisites

- The board is assembled with all the required components and
- The PIC controller has been flashed (e.g. with a PICkit4) with the latest firmware.

2.1 Flashing the "ESP8266 D1-mini" for the first time

There are many ways to flash the ESP for the first time. They all involve connecting the board via the USB interface. The appropriate driver for the USB interface IC installed on the D1 mini may need to be installed first.

Under Linux, the command 'lsusb' can be used to check this. The name of the USB port can be found in the /dev/ directory:

```
~$ lsusb
Bus 002 Device 006: ID 1a86:7523 QinHeng Electronics CH340 serial converter
...

~$ ls -l /dev/ttyUSB*
crw-rw---- 1 root dialout 188, 0 Dez 15 14:00 /dev/ttyUSB0
~$
```

2.1.1 Arduino IDE (2.2.1)

Board: LOLIN(WEMOS) D1 mini
FlashSize: 4 MB (FS: 1MB, OTA: ~1019KB)

2.1.2 tasmotizer

"Tasmotizer" is a Python application and must therefore be started via Python (currently python3). The tool *tasmotizer.py* can be downloaded from github, the path in the following screenshot must be adapted to the download location:

```
~$ python3 .local/bin/tasmotizer.py
esptool.py v2.8
Serial port /dev/ttyUSB0
Connecting....
Chip is ESP8266EX
Features: WiFi
Crystal is 26MHz
Uploading stub...
Running stub...
Stub running...
Configuring flash size...
Auto-detected Flash size: 4MB
Erasing flash (this may take a while)...
Chip erase completed successfully in 3.1s
Compressed 383456 bytes to 276588...
Wrote 383456 bytes (276588 compressed) at
0x00000000 in 24.4 seconds (effective 126.0
kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```



Figure 1: User interface of „Tasmotizer“ (see https://github.com/tasmota/)

In the graphical user interface, simply select the USB port and the binary program (ESP-ValveControl.ino.bin) with "Open" (navigate to the storage location) and press "Tasmotize!". The rest happens automatically.

2.1.3 Command line using "esptool.py"

The Python tool "esptool.py" is usually part of the Arduino IDE, but can also be downloaded and installed from "Espressiv" if Phyton is not already installed on the computer. The path information for esptool.py and the ESP-ValveControl binary file must of course be adapted again:

```
~$ python3 ~/.arduino15/packages/esp8266/hardware/esp8266/3.1.2/tools/esptool/esptool.py -p /dev/ttyUSB0
write_flash 0x00000 ~/.git/OpenValveControl/ESP-ValveControl/build/esp8266.esp8266.d1_mini_clone/
ESP-ValveControl.ino.bin
esptool.py v3.0
Serial port /dev/ttyUSB0
Connecting....
Detecting chip type... ESP8266
Chip is ESP8266EX
Features: WiFi
Crystal is 26MHz
MAC: 24:a1:60:3a:bb:83
Uploading stub...
Running stub...
Stub running...
Configuring flash size...
Compressed 383456 bytes to 276588...
Wrote 383456 bytes (276588 compressed) at 0x00000000 in 24.4 seconds (effective 125.9
kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
~$
```

Figure 2: Using esptool in linux console

2.2 WiFi Configuration

If the flashing was successful, the following message[1] should appear in the first two lines of the **OLED display** (as an indication that the ESP is working as an access point):

```
OVC-Access-Point
OVC-password
```

Shortly afterwards, the message changes and the access point's IP address is displayed:

```
IP: 192.168.4.1
21.1 C          0.0 mA
```

1 If the OLED display does not work, the hardware and especially the connection (and polarity) of the OLED must be checked.

Now you can connect via WiFi to this access point "OVC-Access-Point" and the password "OVC-password" (case sensitive!) and enter the displayed IP address in a browser.

The following message should appear in your browser:



http://192.168.4.1

Figure 3: Appearance in browser prior upload of filesystem user interface

The file "fs.html" must be selected from the ESP directory with "Browse" and then copied to the ESP file system by clicking the button "Upload".

The page then updates itself and shows the ESP8266 file manager with the content "fs.html", the file that has just been uploaded:



http://192.168.4.1/fs.html

Figure 4: Appearance in browser after upload of basic filesystem user interface (fs.html)

In the same way, we then upload the files

"**style.css**" (css formatting for fs.html) and
"**index.html**" (the ESP Valve Control user interface)

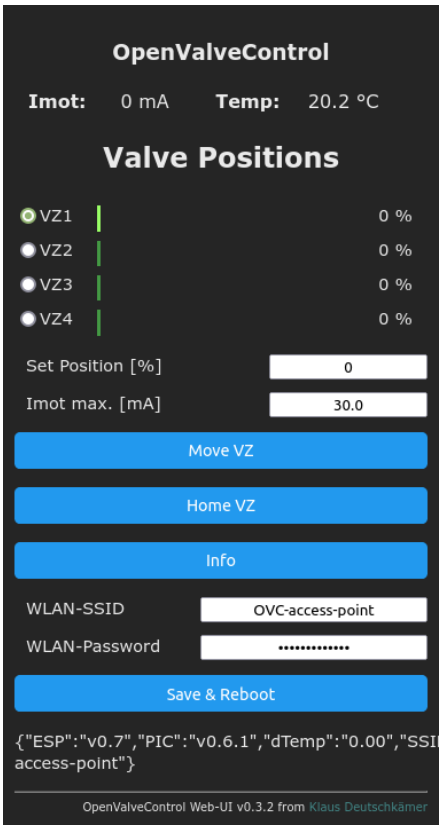
to the ESP file system:



http://192.168.4.1/fs.html

Figure 5: Appearance in browser after upload of fs.html, stylesheet (style.css) and web-UI (index.html)

After entering the top URL (in our example: http://192.168.4.1) into your browser will display the graphical user interface of ESP-ValveControl (it is loaded from the ESP file system):



http://192.168.4.1 (IP of access point)

← Enter here the WiFi SSID of your home network,
← and the WiFi password.
← Press this button to save the settings.

Figure 6: OpenValveControl Graphical User Interface in Web-Browser

2.3 Reboot and Login to your home network

After the reboot, the ESP attempts to log into the home network with the data entered. If this is not successful within about 2 minutes, the access point is activated again.

If problems occur, please check whether your router is configured so that only known devices are allowed to connect. Change this setting temporarily if necessary.

When restarting, the SSID you entered for your home network is displayed in the top line of the OLED. This line flashes while the connection has not yet been established.

The version of the ESP8266 software is displayed in the second line:

Example:

```
FRITZ!Box 6390 Cable
ESP v0.9
```

After successful login, the first row of the OLED now shows the IP address as assigned by your router.

The second line shows the actual temperature[2] and motor current.

Example:

```
IP: 192.168.2.131
19.5 C          0.0 mA
```

You can now connect to the home network and enter the IP address displayed. The ESP-ValveControl graphical user interface should then be displayed again, but now with the SSID of your home network.

2.4 Additional settings (ovc.ini)

After entering and saving the WiFi parameters, an "ovc.ini" file is created in which the settings are saved permanently. Even after a firmware update, the data is normally still available unless you specify that the flash is to be completely erased (e.g. with Tasmotizer: "Erase before flashing").

However, a loss is not a problem, the installation described can be repeated at any time.

An example file **ovc.ini** is available in github in the folder "/ESP-ValveControl/".

```
## ovc.ini
# As default (or when it cannot connect within 2 minutes to the WiFi using the current
# settings), the ESP creates an access point with the credentials shown below.
# Please change the SSID and PSK to meet the settings of your WiFi, then upload the file into
# the ESP's file system, which can be accessed in your browser with the URI <local IP>/fs.html

# WLAN credentials
#SSID = Your SSID
#PSK = Your WiFi password
SSID = OVC-access-point
PSK = OVC-password

# MQTT: server (IP) and token to publish data every period (seconds) as "<mqtt_prefix>/status"
MQTT_HOST   = 192.168.2.72
MQTT_PREFIX = OVC-1
MQTT_PERIOD = 900

# ValveZone aliases (not used yet)
VZ1 = ESS
VZ2 = KÜ
VZ3 = BAD
VZ4 = WZ1

# Adjust temperature sensor
dTemp = -0.5
```

Figure 7: Example file **ovc.ini**

2 If no DS18B20 is connected, -128.2 C is displayed.

It means:

SSID	The name of your home network
PSK	Password of your home network
MQTT_HOST	IP-Adresse of your MQTT server.
MQTT_PREFIX	Prefix for the sent data (to differentiate between several identical devices (all data is sent with the MQTT token "<mqtt_prefix>/status").
MQTT_PERIOD	Time interval in seconds for sending the MQTT data ("publishing").
VZ1 VZ2 VZ3 VZ4	Alternative identifiers for the valve zones (not yet used)
dTemp	Adjustment value for the DS18B20 temperature sensor

As a further alternative, you can also download the „ovc.ini“ file to your computer and save it or change it with a text editor and add further information (e.g. for MQTT).

You can also duplicate the settings on several devices or skip entering the WiFi settings in the access point and upload a prepared „ovc.ini“ file using the Filesystem Manager. After the reboot, all settings are then immediately active.

2.5 Optional JavaScript file (customize.js)

Starting from WebUI v0.4 you can upload a JavaScript file ‚customize.js‘ to the ESP file system. This script can replace some default headers and labels by your own texts, e.g.

- Change title (e.g. 1st Floor, to differentiate between several devices)
- Replace valve identifiers (VZ1, VZ2,..) with more meaningful names.
- Define/translate error messages

The script is read at the end of <head> section in the WebUI’s „index.html“, but the function ***CustomizeLabels()*** is called once at the end of <body>, when the complete HTML document has been read by the browser.

The script also can be extended by your own instructions.

An example file **customize.js** is available in github in the folder "/ESP-ValveControl/".

```
/* customize.js
   Upload this script into the ESP filesystem.
   It replaces some default headers and labels by the given texts.
*/
function CustomizeLabels ()
{
  document.getElementById('header1').innerHTML = '1<sup>st</sup> Floor';   // -> OpenValveControl
  document.getElementById('header2').innerHTML = 'Valve Positions';        // -> Valve Positions

  document.getElementById('radio1').labels[0].innerHTML = '1-Living';      // -> VZ1
  document.getElementById('radio2').labels[0].innerHTML = '2-Dilbert';     // -> VZ2
  document.getElementById('radio3').labels[0].innerHTML = 'VZ3';
  document.getElementById('radio4').labels[0].innerHTML = 'VZ4';
}

// Error messages may be changed or translated if required.
// When you change the no. of items in this array, don't forget to adjust
// the function f_status() in index.html!
const ErrorMessage =
[
  "AD converter timeout",                 // -127   PIC errors
  "EEPROM write: argument count != 2",    // -9
  "EEPROM write address out of range",    // -8
  "MOVE command when HOME is active",     // -7
  "MOVE command: argument count != 3",    // -6
  "Reference not set",                    // -5
  "Undefined command",                    // -4
  "Out of range: mA_max",                 // -3
  "Out of range: set_pos",                // -2
  "Out of range: vz",                     // -1
  "No error",                             //  0
  "INI file not found",                   //  1     ESP errors
  "Identifier not found",                 //  2
  "PIC timeout",                          //  3
  "Move not acknowledged by PIC",         //  4
  "Home not acknowledged by PIC",         //  5
  "Status not acknowledged by PIC",       //  6
  "EEPROM write not acknowledged by PIC", //  7
  "EEPROM write: address out of range",   //  8
  "EEPROM write: argument count != 2",    //  9
  "EEPROM write: data out of range"       // 10
]
```

Figure 8: Example file **customize.js**

As an example, you will find the first replacement item in the <body> section of „index.html“:

```
<h2 id='header1'>OpenValveControl</h2>
```

The header text „`OpenValveControl`“ will be replaced with the given text in the script, because the `<h2>` expression in „index.html“ has the attribut `id ='header1':`

```
<h2 id='header1'>1<sup>st</sup> Floor</h2>
```

As you see, you can even enter html attributes like superscript <sup> (superscript) to get „1st Floor“ or similar HTML expressions.

Hence, basically this script overwrites the default headers to differentiate several devices and to give the valve radio buttons more meaningful (room) names.

These texts are preserved even if the file „index.html“ is updated.

2.6 First driving trials

A connected motor without a valve can move beyond its end position if it does not experience any resistance when moving. This may result in damage to the mechanics. The control unit is operated at your own risk.

2.6.1 Delivery status of the drives

The following photo shows an actuator in its as-delivered condition. You can see that the round piston (in the center) is offset a few millimeters inwards. This corresponds to the OPEN valve position on my Rotex heating circuit manifold and makes it easier to install the motor.

When referencing (homing), the actuator moves the piston out, so when installed it pushes the valve pin in, which corresponds to the CLOSED valve position (on my Rotex heating circuit manifold). When the valve is closed, the piston presses against the spring force of the valve pin, which is why installing the motor in motor position CLOSED is not recommended.

This may be different with other devices!
OpenValveControl does not currently allow the direction of rotation to be changed!



Figure 9: As-delivered condition

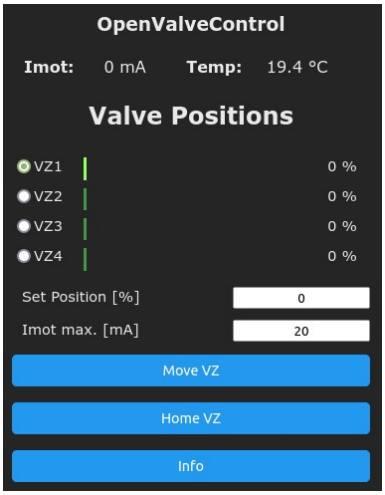
2.6.2 Checking the reference run

First of all, you should carry out a reference run - WITHOUT installing the motor in the heating circuit manifold. The easiest way to do this is on the desk. Connect the motor to the left-hand socket, for example, as shown in the next picture. This corresponds to valve zone VZ1 in the user interface.



Figure 10: Connection of a motor as VZ1 to test the reference run.

- Now select „VZ1“ in the user interface (by clicking with the mouse)
- Then enter the (small) value 20 for „Imot max [mA]“
- Now click on the "HOME VZ" button.



The motor should then move the piston "out" and display the motor current during this process. The LED on the PCB also lights up as an indicator.

The reference run is terminated when the motor current exceeds the entered limit value. Valve position "CLOSED" is then assumed and the actual position is set to 0 %.

You can easily simulate this with such a low limit value by pressing against the piston with a suitable tool or with your thumb while it is being moved out.

Figure 11: Graphical User Interface

Caution! The piston must not move out so far that the guide lug comes completely out of the groove, otherwise the piston cannot be moved in without manual support.

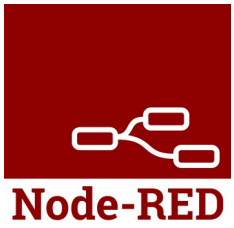
The reference run is therefore aborted after approx. 2 minutes, even if the current limit value is not exceeded. The LED goes out.

In case of problems, simply unplug the drive!

If this "reference run" has worked, you can specify a „Set Position“ of e.g. 20 % for the motor and move to this position using the "Move VZ" button.

If the motor stops prematurely, you can increase the current limit value in small steps. Proceed in a similar way once you have installed the motor. For comparison: I have entered a current limit value of 55 mA for my Rotex heating manifold.

3 Controlling with node-RED[3]



Node-RED is the "Swiss army knife" of the Internet of Things (IoT) and the ideal tool when it comes to exchanging data between sensors and actuators. In this chapter, we want to control *OpenValveControl* with a node-RED „flow“.

In node-RED, "flows" refer to the graphical representations and data connections of "nodes". A node can, for example, represent an interface to MQTT and is represented as a block (node) with inputs and outputs. The properties of these nodes are defined via parameters (e.g. the IP address of the MQTT server).

You can install node-RED on a RaspberryPi, for example, so that it is permanently available (24/7). On the same RaspberryPi, a mosquitto server can run in parallel as an MQTT host and other applications (e.g. grafana).

I have been running this configuration on a 3rd generation Pi for years without any problems.

3.1 Adjusting the valves using node-RED

3.1.1 Task definition

As we have seen, we can adjust our valves via the web interface. To do this, we need to know the IP address of the web UI.

Then we can select the ValveZone (the actuator or valve), enter the target position (0 to 100%), enter the maximum motor current in milliamperes and press the "Move" button.

This sends an "http GET" command to the ESP, which contains the parameters mentioned. The command is displayed in plain text at the bottom of the Web UI screen.

Example

Valve 1 (vz=1) should be moved to 26%. If the motor current exceeds the value 30 mA, the process should be aborted (blocking protection):

```
/move?vz=1&set_pos=26&max_mA=30.0
```

We can achieve exactly the same thing by sending the GET command via a web browser on our LAN. We just need to prefix it with the IP address of our control device (the IP address will be displayed on the OLED after the ESP has successfully logged into the home network):

```
http://192.168.2.131/move?vz=1&set_pos=26&max_mA=30.0
```

3 **node-RED** is a flow-based programming tool, originally developed by IBM’s Emerging Technology Services team (https://emerging-technology.co.uk/) and now a part of the OpenJS Foundation (https://openjsf.org/).

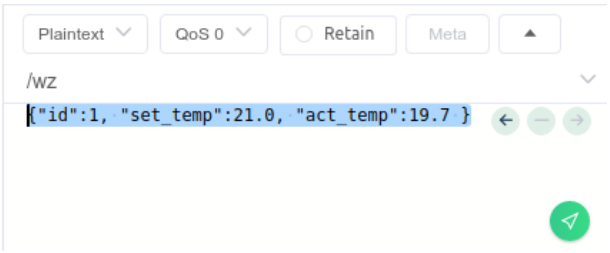
3.1.2 Specifying a setpoint

In order to solve the task in a practical way, we first have to specify the target value.

We assume that we have a temperature sensor with display in our living room for a reasonable individual room control as well as a possibility for a setpoint specification. This could be realized with a small ESP or with a commercially available industrial product (display of actual and setpoint temperature, 2 buttons ⇅ for the setpoint).

This ESP could now send the described GET command directly to our controller via WiFi and, for example, control the valve to 80% if it is too cold in the room or to 20% if it is too warm (simple "two-point" control). In addition, each thermostat must of course be informed of the valve assignment, the maximum current and possibly the control algorithm. However, there is other information that is useful for control, e.g. outdoor temperature, solar radiation, night setback, extended absence (vacation) and others. It therefore makes sense to only report the actual and setpoint temperatures of the respective room and to manage all rooms centrally, e.g. with node-RED.

To do this, we send a sensor ID, the actual temperature and setpoint value to an MQTT server (also known as a broker, e.g. mosquitto). Until we have programmed the sensor, we can simulate this with the MQTTX[4] tool:



If we are connected to the MQTT broker (e.g. as test@raspi3:1880), we can define a **topic** (e.g. "/wz") and send the entered **payload** to the MQTT broker by clicking on the green arrow. The data can be changed as required.

Figure 12: Using MQTTX to simulate data source

3.1.3 flow: "Subscribe" thermostat values from our MQTT broker

In node-RED, we now place an "mqtt in" node in our "flow". Double-clicking on the node opens the "Properties" window, in which we enter the following data:

I have entered "localhost:1883" as the **server** because both node-RED and mosquitto (the MQTT broker) run on the same RaspberryPi. Otherwise, the corresponding IP address of the MQTT server must be entered instead of "localhost".

4 „Your All-in-One MQTT Client Toolbox“ (https://mqttx.app/). MQTTX is a free open-source project available under the Apache-2.0 LICENSE.

We can assign the **topic** "/wz" (deutsch: WohnZimmer = Living Room) in the thermostat (or temperature sensor). We have entered this in MQTTX as a test and it is used to differentiate between the various rooms or temperature sensors.

Quality of Service "**QoS**" describes the increasing degree of reliability in the delivery of messages, the highest value is 2.

As **output**, we want a "parsed JSON object", i.e. the same message that was sent from the sensor to the MQTT broker.

The **name** can be chosen at will and is primarily used to make the flow clearer.

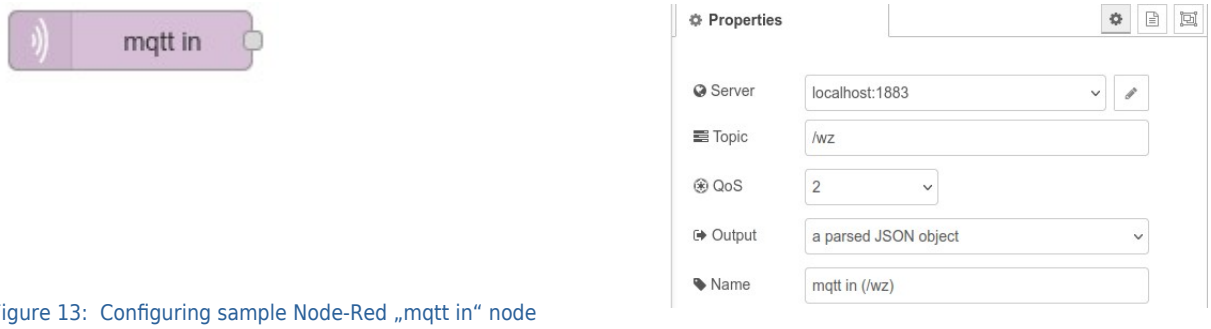


Figure 13: Configuring sample Node-Red „mqtt in“ node

We now add a "debug" node so that we can check the output of the "mqtt in" node. As soon as we send a data record to the MQTT broker via MQTTX, the subscribed data is sent to node-RED and is available at the output of the "mqtt in" node:

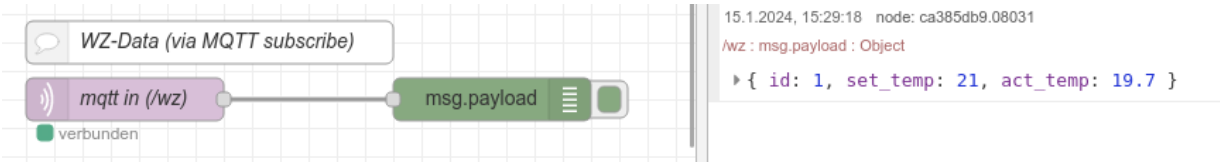


Figure 14: Using Node-Red „debug“ node

3.1.4 flow: Convert temperature difference into valve position

We now know the room ID, the setpoint and actual temperature and can now make a decision for the valve position.

To simplify matters, we take the room ID as the valve zone (vz) and program the two-point controller described above, which we want to formulate in JavaScript as follows:

If the deviation is more than 0.5 degrees, we switch the valve closed (20%) or open (80%), otherwise we leave it as it is.

Note: The simplified assumption is that the flow temperature is weather-compensated and we only make a fine adjustment with a valve. Of course, we can also program an arbitrarily complicated control algorithm here!

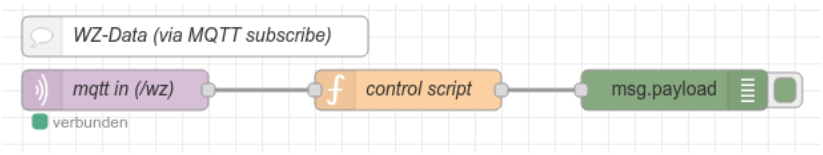
For this purpose, node-RED has the "function" node, in which JavaScript can be executed almost arbitrarily. A "message" is accepted as input, which can be modified (almost) at will by the program and output at one or more outputs. The instruction `return (x)` is used for this purpose, whereby x can also be `null`, in which case no output is generated. We use this hysteresis to avoid unnecessary adjustments if the temperature is reasonably correct.

At the start of the JavaScript program, our message has the following values, for example:
`msg.payload = {"id":1,"set_temp":21,"act_temp":19.7}`

At the output, we want to have the appropriate parameters for the "http GET" request:
`msg2.payload = {"id":1, "set_pos":80, "max_mA":50.0}`

All we have to do is generate a new message msg2. We temporarily take the missing value for "max_mA" from a variable that was somehow saved in the flow with the instruction `flow.set("max_mA", <value>)`, for example.

Our JavaScript program could then look like this:



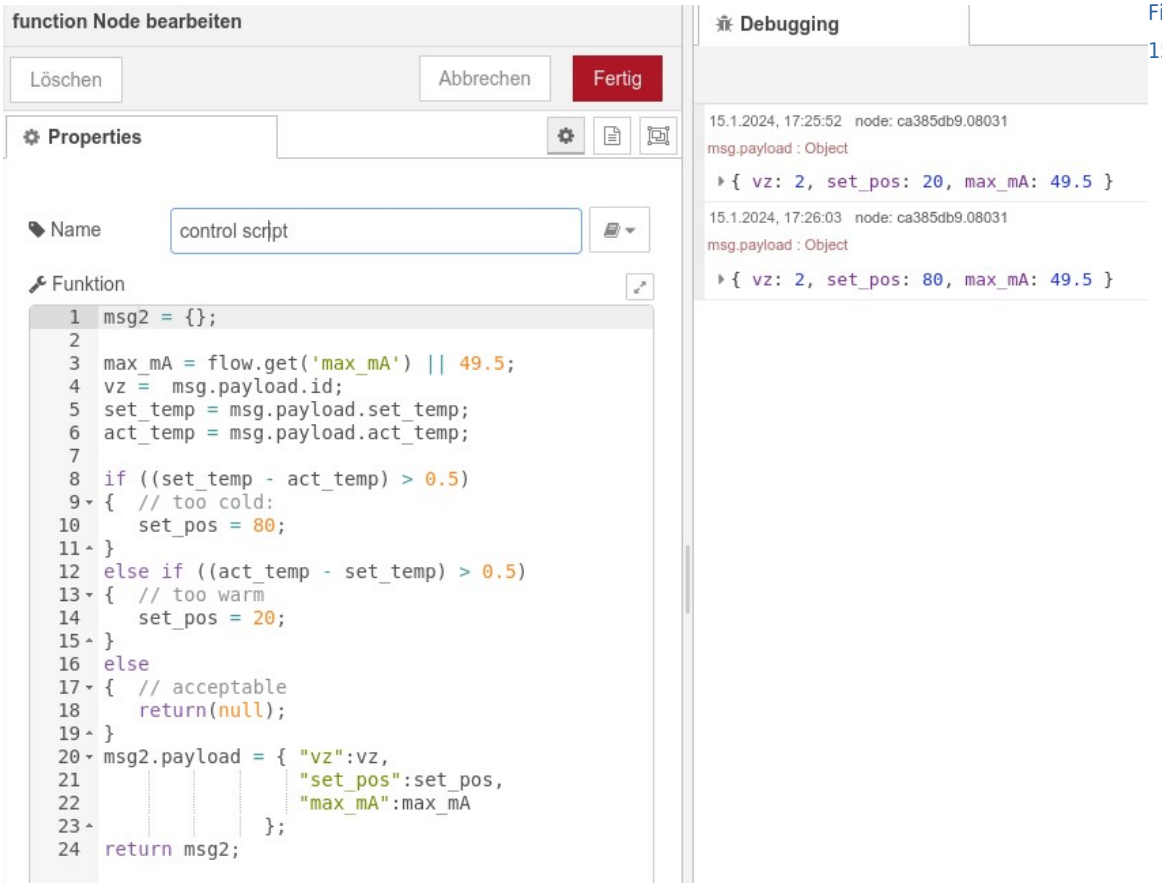


Figure 15: Configuring sample Node-Red „function“ node

The 2 outputs in the debugger were the responses to the following 3 messages:

```
{"id":2, "set_temp":18.0, "act_temp":19.7 }        // too warm
{"id":2, "set_temp":22.0, "act_temp":19.7 }        // too cold
{"id":2, "set_temp":20.0, "act_temp":19.7 }        // acceptable
```

This is a very good way of testing that no output is produced on the third input as intended.

Note

To prevent an error message from occurring when reading the max_mA variable or an invalid value being used if no value has (yet) been saved, a logical AND has been appended, which assigns a default value in this case:

```
max_mA = flow.get('max_mA') || 49.5;
```

3.1.5 flow: Output the query parameters as http GET request

There is another (network) node for sending the "http request". We place it at the output of the JavaScript "function" node and connect both nodes. We move the debug node to the output of the http request. Then we fill in the properties as follows:

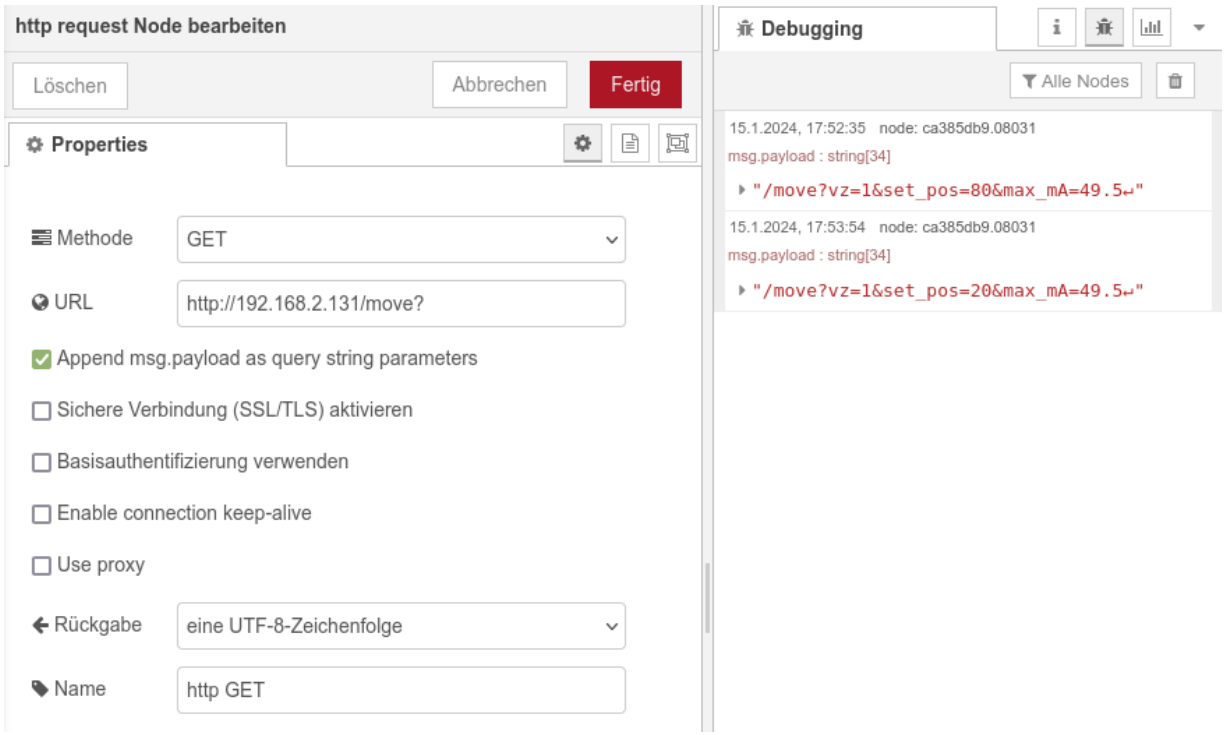


Figure 16: Configuring sample Node-Red node „http request“

The debug output when publishing our thermostat values shows the response of our ESP server, when it receives the GET request. In our case, fortunately the ESP echoes back the request (watch the CR ↲ at the end of the debugged payload). So we can see that all parameters from the message were appended to the URL with the addition "/move?" for the move command.

Similarily, the commands "home" or "info" or "status" can also be implemented and further processed in node-RED.

Of course, you can delete or deactivate the debug node if you no longer need it.

This completes the task and we have our finished flow:

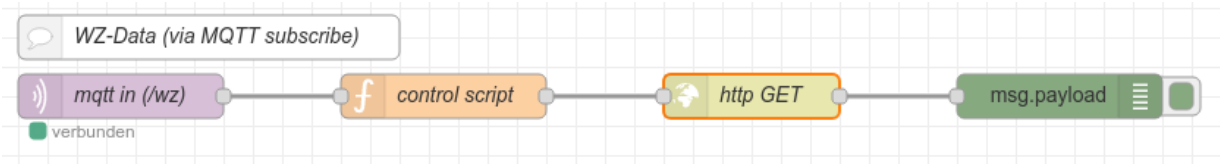


Figure 17: Complete sample in Node-Red for valve control

4 Driving the motors by PWM

4.1 Introduction to the H-bridge

In our application we are using two „MX1508 Brushed DC Motor Drivers“. Each board can drive two motors independently. So we can run up to 4 motors in total.

These drivers incorporate an H-bridge for each motor and can drive the motors in both directions. A simpified schematic of each H-bridge is shown in the following figure:

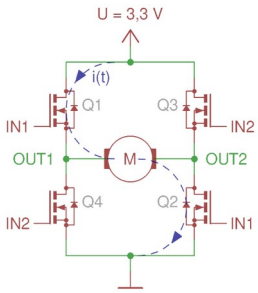


Figure 18: Simplified schematic of each H-bridge

The motor is connected to outputs OUT1 and OUT2.

When IN1 is high and IN2 is low, a current flows from the voltage source through Q1, the motor and through Q2 to GND, moving the motor in one direction.

When IN1 is low and IN2 is high, the current flows from the voltage source through Q3, the motor and through Q4 to GND, moving the motor in the opposite direction.

If both inputs are low, all transistors are high-impedance and no current flows.

Internal logic inhibits a short between U and GND, when both inputs are high. Instead, transistors Q1 and Q3 are switched off, Q2 and Q4 are switched on, shorting the current through the motor. This has a similar effect as a brake.

To drive a motor in one direction, instead of a constantly high level we also can apply a pulse width modulated (PWM) signal. In this case, we are able to control the motor power p(t) = u(t) * i(t) by changing the duty cycle of the PWM signal.

But there is yet another advantage: During the period in which the H-bridge is switched off, we want to measure the motor’s „Back EMF“ (Electro Magnetic Force), a voltage almost proportional to the motor speed, which may be summed up as a position estimation.

The PWM signal can also be used to trigger an interrupt for the acquisition of the motor current, which is required to switch off the motor in the event of a blockage.

4.2 PWM generation and interrupts

After a couple of experiments we finally generate three synchonized PWM signals with the PIC's PWM1 and PWM2. Each PWM instance of the PIC18F16Q41 can generate one "slice" called S1, whose behavior can be defined by 2 parameters P1 and P2. Accordingly, the three signals used are named *PWM1S1P1, PWM1S1P2* and *PWM2S1P2*, following the naming convention of the PIC datasheet.

PWM1S1P1 is used to enable the H-bridge. The signal can be output to any arbitrary port pin by writing the value 0x0A to the pin's corresponding RxyPPS (peripheral pin select) register. By this we can assert the signal to either IN1 or IN2 of the motor driver module, depending on the desired motor direction. The width of the High-level was set to 7.2 ms with PWM1 parameter 1 followed by a 800 µs L-level. At the L-level, no current is driven through the motor and so the motor acts like a generator where we can measure the back EMF voltage (V_{BEMF}).

PWM1S1P2 is used to measure the motor current. After switching on the H-bridge, we have to wait until the current has reached a stable value. Therefore, the width of the H-level was set to 2 ms with PWM1 parameter 2 and the falling edge triggers the PWM1 interrupt. The interrupt service routine then reads the motor current.

PWM2S1P2 is used to measure the back EMF voltage (V_{BEMF}). similarily, after switching off the H-bridge, we have to wait until V_{BEMF} has reached a stable value. Therefore, the width of the H-level was set to 7.6 ms with PWM2 parameter 2. The falling edge triggers the PWM2 interrupt. Accordingly, the interrupt service routine then reads V_{BEMF} .

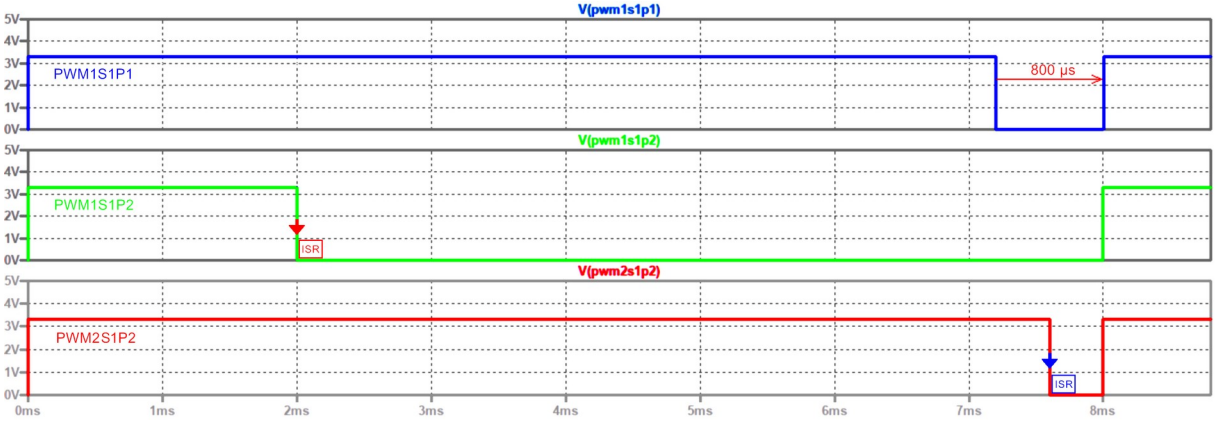


Figure 19: Timing of PWM „slices" (LTspice simulation)

4.3 Acquisition of motor current

As shown in the schematics, the motor current is measured by an INA219 breakout board, where the small voltage drop across a shunt is sensed and can be read via the I2C interface. Our INA219 board uses a 0.1 Ohm shunt, but may be different on others. 100 mA motor current would then generate a voltage drop of only 10 mV. Due to the resolution of 10 µV, current changes of 0.1 mA can be measured.

These defaults are working pretty good for our application, as we can simply read the INA216 register at address 1, which continuously is updated with the shunt voltage measurement data using the default configuration. No further initialisation or configuration is required.

The shunt voltage register is in signed 16 bit format and LSB = 10 µV, which is equivalent to 0.1 mA (I = U / R).

As previously explained, we read the motor current in the interrupt service routine, on the falling edge of our PWM signal *slice1, output2 (SaOUT2)*. This is 2 ms after the H-bridge is switched on and allows the current to stabilize.

To read INA216 register 0x01, we first have to write the desired register address (0x01) to the configuration register at address 0x00. After this, we can start two I2C read cycles from the INA219. The most significant byte (MSB) is read first.

This measurement is carried out by the PIC with a I2C clock of 400 kHz. Address setup and 16 bit read of the shunt voltage register require 138.5 µs (add some microseconds for data handling and cleanup).

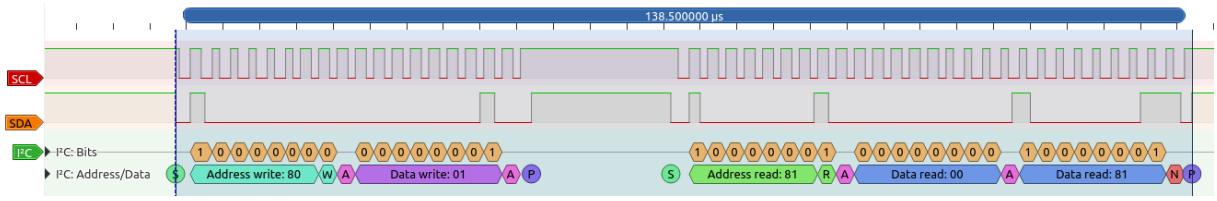


Figure 20: Visualisation of this I2C communication recorded with a logic analyzer

The reading is repeated every PWM cycle (8 ms). But we also want to display the current at standstill, so we must also read the current when the PIC is in the 'idle' state.

These are the function calls to read the motor current from INA219 module:

```
/* Read motor current via INA219 Shunt Current Register
 * (1 LSB = 10µV, Rs = 0,1 Ohm => I / 0.1 mA = Us / 10µV).
 * Exectime ~171 µs.
 */
ina219_reg(1);                // exec time ca. 49 µs (@SCL 400 kHz)
g_mAx10 = ina219_read();      // exec time ca. 72 µs (@SCL 400 kHz)
if (g_mAx10 < 0) g_mAx10 = 0;  // offset may cause negative readings
```

4.3.1 Exemplary measurement results

The following oscillograms have been recorded using the PIC's digital to analog converter (DAC). This DAC has only 8 bit, and the motor was expected to reach up to 100 mA, which produces a (16 bit) register value of up to 1000 (x 0.1 mA). So the read 16-bit value was shifted 2 bit to the right, which corresponds to a „division by 4" and truncated to 8 bit.

100 mA → 1000 = 0x3E8, divide by 4 → 0xFA (unsigned) = 250

This value results in U_{DAC} = 3.3 V * 250 / 255 = 3.24 V at the output.

Accordingly, we obtain 30.9 mA/volt in the oscillogram..

These are the results in both directions, with no load (free running on the desk) and with some load (applied with my thumb pressing onto the piston).

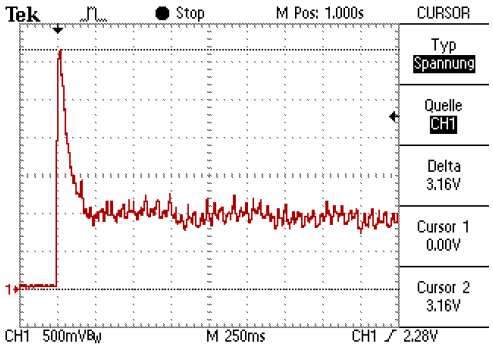


Figure 21: Motor current, open direction, with load

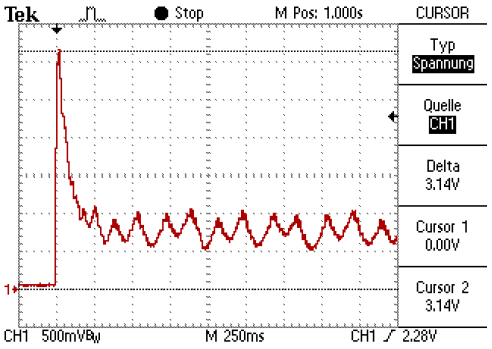


Figure 22: Motor current, open direction, no load

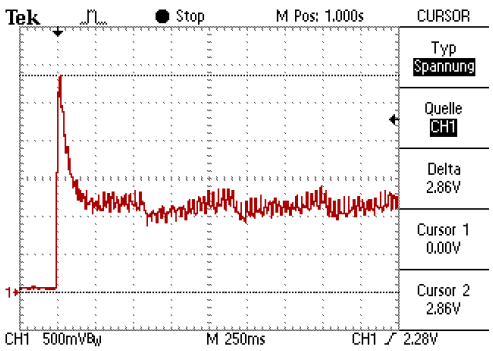


Figure 23: Motor current, close direction, with load

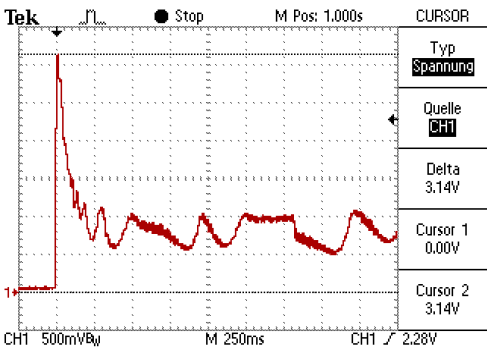


Figure 24: Motor current, close direction, no load

As result we can see, that the motor start-up is completed after approx. 200 ms, causing a peak current close to 100 mA. Without load the motor draws approx. 15 to 30 mA, with moderate load approx. 10 mA more. In reality, when the motor runs against a resistance, the current consumption may increase, hence with a ohmic resistance of R = 30 Ohm and U = 3.3 V - 2 x U_{DS} (H-bridge), the current cannot exceed 100 mA, which is well below the max. current of 120 mA of the VDMOT. And as soon as the motor turns, the back EMF counteracts the supply voltage until the system reachs a steady state.

4.4 Measuring the motor inductance

In the previous chapter we measured the motor current 2 ms after the H-bridge has been switched on. We have seen that the motor current decreases when the motor is turning, because the generated Back EMF (U_{BEMF}) is proportional to the motor speed. Ignoring the effects of brushes and commutation, this can be approximated by the differential equation:

$$L \frac{di(t)}{dt} + R\, i(t) = V - k_e \omega \qquad \text{(Eq. 1)}$$

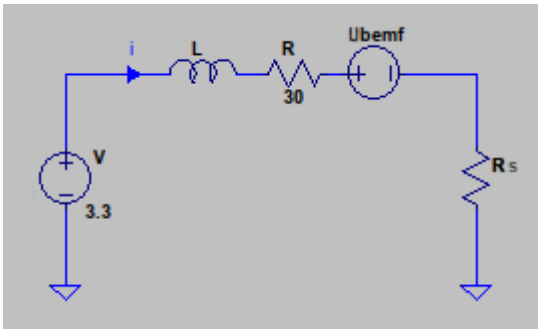


Figure 25: Basic circuit for Brushed DC Motor

Rs = 0 (preliminary)

$U_{BEMF} = k_e\, \omega$

The resistance of the motor coil can easily be measured using an ohmmeter. We got approx. 28.5 ohms, let us round this to 30 ohms to include some contact resistance:

$$R = 30\ \Omega$$

To measure the inductance, we can record the motor current when switched on at standstill because then V_{bemf} is negligible. As a first step we measured the maximum motor current, as this value is required to compute the time constant of an exponential curve.

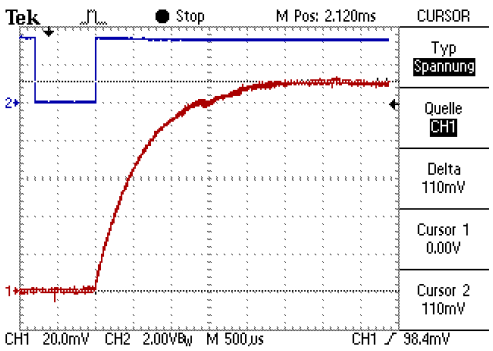


Figure 26: Max. motor current

max. motor current: 110 mA

CH1: Motor current x R_{shunt} (1 Ω)

CH2: PWM

The time constant T = L/R ist the time it takes for the current to reach 63% of its maximum value. In our case this is approx.

T = L / R = 568 µs (Eq. 2)

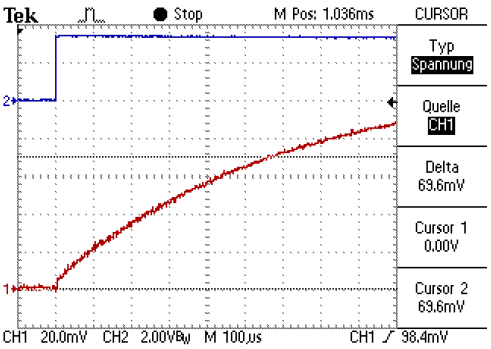


Figure 27: Cursor at approx. 63 % of max current

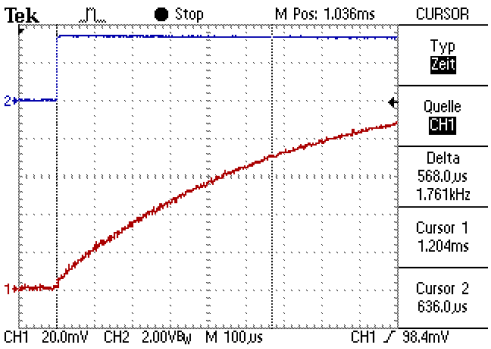


Figure 28: Cursor at time to reach 63 % of max current

We used a 18650 lithium accu with 3.3 V voltage regulator, a low resistance FET as a switch and a 1 Ω shunt resistor to measure the current, which we must add to the coil resistance. Ignoring the resistance of the voltage source, we get

L = 31 Ω · 568 µs = 17,6 mH

L = 17,6 mH

We can also see that reading the motor current 2 ms after switching on the H-bridge is equivalent to

2 ms / 568 µs = 3.5

As the motor current follows the equation

$$i(t) = I_{max}(1-e^{-t/T}) \quad , \qquad \text{(Eq. 3)}$$

after this period, the exponential curve has reached approx. 97 % of its final value.

4.5 Measuring the motor speed

Opening the VDMOT housing, we can see the gearbox with 5-stage transmission. A further stage converts the rotation into a linear stroke (without image):

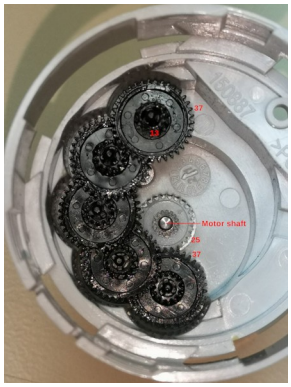


Figure 29: Gearbox of VDMOT

The first gear stage reduces the motor speed n_M by a factor of 25:37. There are 4 further identical stages with gear ratios of 13:37. So the speed n_5 of the last gear wheel is

$$n_5 = \frac{25}{37}\left(\frac{13}{37}\right)^4 n_M = \frac{714025}{69343957}\, n_M = 0{,}010297\; n_M$$

By taking a video capture, we counted 19 revolutions of the last gear wheel within 44 seconds. So we can calculate the **motor speed without load** as:

$$n_m = \frac{1}{0{,}010297}\,\frac{19}{44\,s} = 41.937\,s^{-1}$$

$$\boxed{n_m = 2516\; min^{-1}}$$

In this case, the motor is running with absolutely no load, but let us remember that one motor revolution takes approx. 24 ms, which corresponds to around 3 PWM periods.

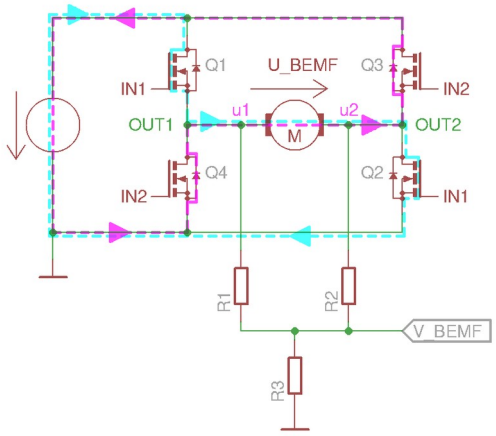
4.6 Measuring the back EMF

Each turning motor also acts as a generator. The generated voltage is called the Back Electro Magnetic Force U_{BEMF} and is proportional to the motor speed. This voltage can be measured, when the motor is free running, that is, when the H-bridge is switched off (at least one motor terminal).

An optimum solution would use a differential amplifier for each motor to measure (u1 - u2) at the motor terminals. Because we want to measure both driving directions, this difference will have a negative sign in one direction which adds some complexity.

Another method could measure U_{BEMF} when one side of the motor is floating and the other is grounded. Hence, the given break-out board doesn't allow this combination.

A simple solution[5] uses a voltage divider with two symmetrical inputs to measure the sum (u1 + u2) as shown in the next figure. This circuit works for both directions of motor rotation and requires only 3 resistors (R1 to R3) per motor:



cyan: H-bridge is enabled, motor is driven by supply voltage and flows through enabled FETs Q1 and Q2.

pink: H-bridge is disabled, motor current is driven by inductance and flows through bypass diodes.

Figure 30: Measuring back EMF voltage

As soon as the H-bridge is switched off, the inductance tries to maintain the current in the motor, until the energy stored in the coil is dissipated. The current cannot flow via the blocked FETs, instead it takes the path via the bypass diodes (see the pink dashed line in the figure). Because of the diodes, the potential u2 is therefore not GND, but has an offset $-V_f$ and Potential u2 cannot exceed the supply voltage by more than $+V_f$.

The following applies to the voltage divider with 2 inputs:

$$V_{BEMF} = \frac{u_1}{1+\frac{R_1}{R_2}+\frac{R_1}{R_3}} + \frac{u_2}{1+\frac{R_2}{R_1}+\frac{R_2}{R_3}} \qquad \text{(Eq. 4)}$$

5 Texas Instruments Article „Easy cruise control for brushed motors using BEMF“ by Mark Buccini, 2013 https://e2e.ti.com/blogs_/b/industrial_strength/posts/easy-cruise-control-for-brushed-motors-using-bemf (deleted).

As we should treat both directions of rotation equally, we can set R1 = R2.

The equation shows that for the highest possible output voltage V_{BEMF}, the resistance R3 should be significantly greater than R1 so that R1/R3 becomes negligible.

Because this voltage divider forms a bypass for the motor current, the current through the voltage divider should be considerably lower than the motor current, i.e. it should be well below 1 mA.

However, the current through the voltage divider should not be too small as otherwise the charging current of the hold capacitor in the ADC will distort the voltage divider.

Choosing R1 = R2 = 4k7 and R3 = 10k we get

$$V_{BEMF} = 0{,}405\ (u1 + u2) \qquad \text{(Eq. 5)}$$

When the H-bridge is blocked, the voltage u1 (now on the low side), is determined by the forward voltage[6] of the bypass diode at Q4. The anode of the diode is connected to GND, the voltage at the cathode is u1. Therefore

$u1 = - V_f$

Negative voltages at ADC inputs are clamped by diodes within the PIC, resistors R1 and R2 must limit the clamping current to below 20 mA (absolut maximum ratings). Since we cannot measure negative voltages with the ADC, the following must apply:

$u2 > V_f$

$u2 - u1 > V_f + V_f$

$$U_{BEMF} > 2\ V_f \qquad \text{(Eq. 6)}$$

i This means that we cannot expect a useful EMF voltage at very low speeds. Only at medium speeds can the EMF be used for speed estimation.

As our project is designed to be as simple as possible, we do not use differential amplifiers and only use the back EMF for stall detection and speed estimation from medium speed. The motor ramps up so fast, that the position error is much less than the error using a fixed time interval.

It is generally recommended to open and close the valves once a week to prevent limescale build-up. The reference run carried out during this process resets any position difference.

6 We measured a forward voltage of approx. 0.5 V to 0.6 V on the MX1508.

4.6.1 Comparing Simulation and Measurement

When we measured the output signal V_{BEMF} of the voltage divider of our very first PCBs with an oscilloscope, we recorded the following signal course:

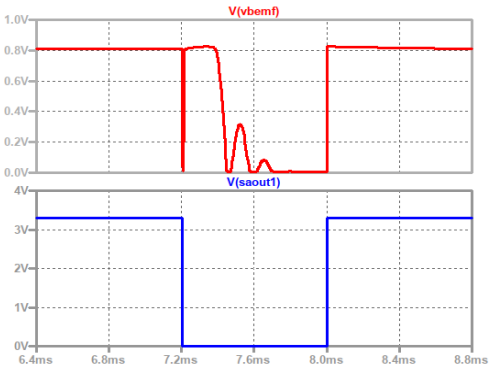


Figure 31: LTspice simulated V_{BEMF} (U_{BEMF} = 0)

V_{BEMF}

PWM

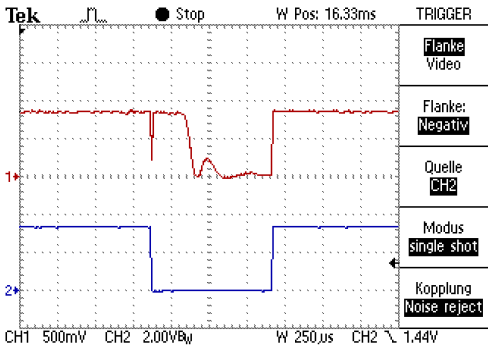


Figure 32: Measured V_{BEMF} (U_{BEMF} = 0), first PCB design

C1 = 100 nF, R1 = R2 = 1 k, R3 = 500

One cause was that the interference suppression capacitor (C1 to C4[7]) formed a resonant circuit with the motor inductance. New recommended values are 10 pF or simply omit these capacitors.

Next, the voltage divider on the first PCB version was realized using resistors 2 x 1k and 500 ohms, which meanwhile (since PCB rev. A) has been changed to 2 x 4k7 and 10 k.

Here is the result with the new component values:

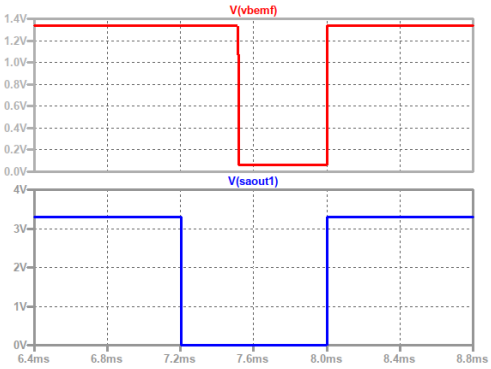


Figure 33: LZSpice simulated V_{BEMF} (U_{BEMF} = 0), C1 = 10 pF

V_{BEMF}

PWM

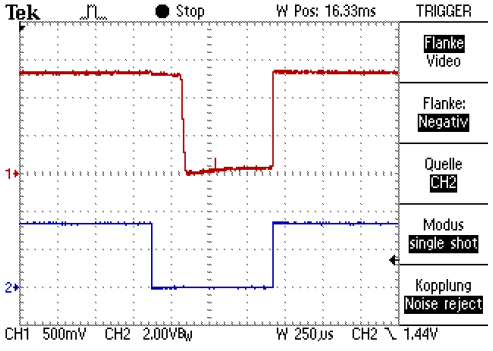


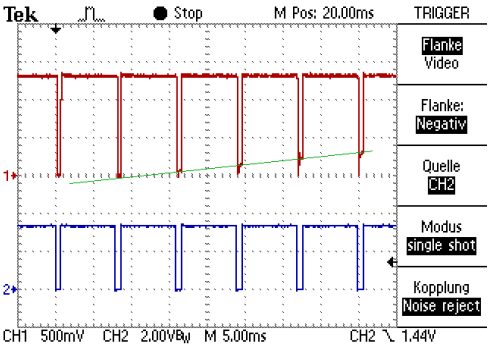
Figure 34: Measured V_{BEMF} (U_{BEMF} = 0), C1 = n.a.

new attenuator: R1 = R2 = 4k7, R3 = 10k

When disabling the H-bridge, there is no more V_{BEMF} oscillation and V_{BEMF} is nearly stable after approx. 320 µs. The voltage level has increased significantly from 0.8 to 1.3 V.

7 In the first design, these capacitors had 100 nF, which led to the oscillations.

The next oscillogram on the left shows the first 6 PWM cycles with an increasing V_{BEMF} signal. The oscillogram on the right is a magnification of the 6th V_{BEMF} measurement.



V_{BEMF}

PWM

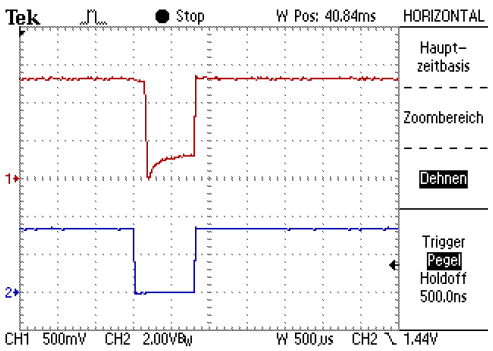
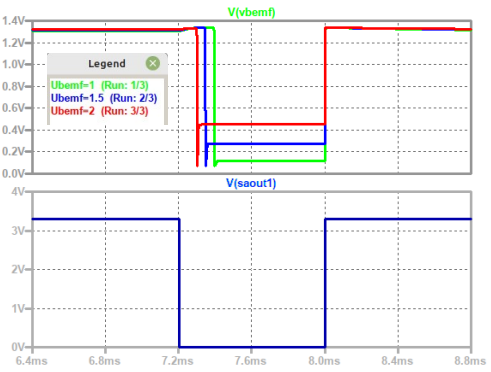


Figure 35: Increasing V_{BEMF} (first 6 PWM cycles)

Figure 36: Zoomed V_{BEMF} (last PWM cycle)

The measurement must be delayed until the energy stored in the inductance has dissipated. The simulation shows, that this settling time becomes shorter as the BEMF voltage increases:



V_{BEMF}

PWM

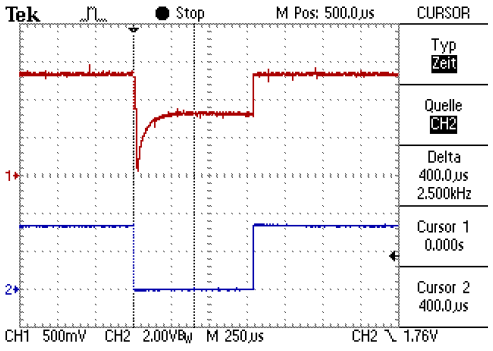


Figure 37: V_{BEMF} (LTSpice STEP: 1 V, 1.5 V, 2 V)

Figure 38: Measured V_{BEMF} (max speed, with load)

V_{BEMF} in the real measurement rises significantly slower than in the simulation. The reason for this behaviour could not yet be determined, but is probably caused by the components on the H-bridge break-out board.

4.6.2 Back EMF sample oscillograms

Now lets see, what our PIC controller reads when moving position from 0 to 20%.

The following oscillograms have been recorded using the PIC's digital to analog converter (DAC). U_{BEMF} is expected to reach max. 4 V, which is reduced by the voltage divider to approx. (V_{BEMF} = 0.4 * 4 V = 1.6 V). This value is read by the PIC's 12 bit ADC using a 2.048 V reference voltage, resulting in ADRES values of approx. 4095 * 1.6 V / 2.048 V = 3199 = 0xC7F. Because the DAC has only 8 bit, the read 12-bit value was shifted 4 bit to the right, which corresponds to a „division by 16":

4 V (U_{BEMF}) → 1.6 V (V_{BEMF}) → ADRES = 3199 = 0xC7F, div. by 16 → 0xC7 = 199 (DAC)

This value results in U_{DAC} = 3.3 V * 199 / 255 = 2.57 V at the DAC output. Accordingly,
1 Volt in the oscillogram corresponds to 1.55 volts of the back EMF.

These are the results in both directions, with no load (free running on the desk) and with some load (applied with my thumb pressing onto the piston):

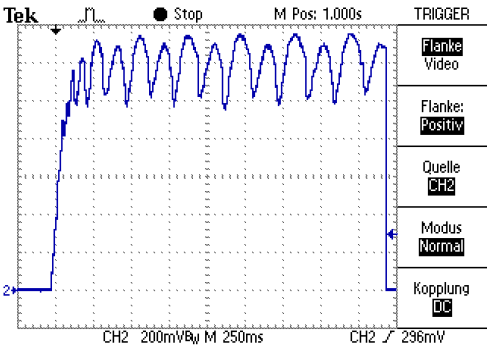


Figure 39: Back EMF voltage, 0 to 20%, no Load

V_{BEMF}

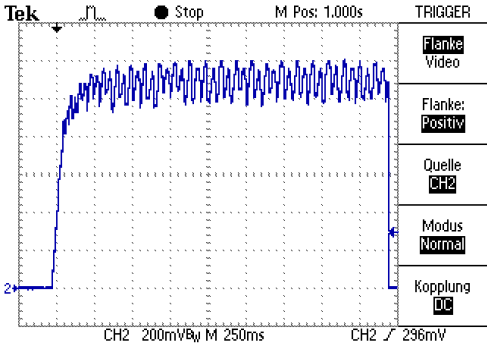


Figure 40: Back EMF voltage, 0 to 20%, with Load

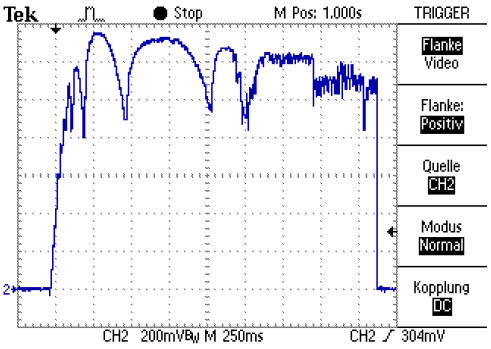


Figure 41: Back EMF voltage, 20 to 0%, no Load

V_{BEMF}

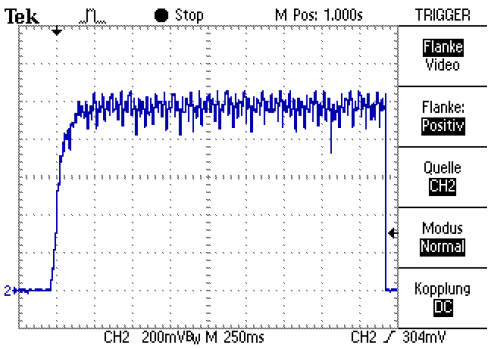


Figure 42: Back EMF voltage, 20 to 0%, with Load

Note: When a debugger is connected (ISPDAT/ICSPCLK), V_{BEMF} of VZ1 and VZ2 cannot be measured. The PICkit4 can also cause a voltage offset when not properly grounded. The signal shape also depends from the motor, maybe the gear is making the difference.

As expected we see, that the speed depends on the load and the direction. The shape of the curve is probably mainly influenced by the friction of the gearbox. With more load, the speed gets lower. The time to stop the motor is nearly zero, in contrast to the start-up which takes approx. 150 ms.

To get an estimation of the position, we simply can add up all the V_{BEMF} readings, hopefully giving a better position control than using constant time intervals.

4.7 Calculating the motor constant k_e

Assuming that max. V_{BEMF} = 1.4 V (no load) corresponds to max speed n_m = 42 s^{-1}, we can compute the motor constant k_e as follows:

$$U_{BEMF} = V_{BEMF} / 0.404 = k_e \, \omega = k_e \, 2 \, \pi \, n_m$$

$$k_e \approx 1.4 \text{ V} / (0.404 \cdot 2 \, \pi \cdot 42 \text{ s}^{-1})$$

$$\boxed{k_e \approx 13.13 \text{ mV/ s}^{-1}} \qquad \text{(Eq. 7)}$$

And with a given V_{BEMF} we can compute the motor speed:

$$\boxed{n_m = V_{BEMF} / (0.404 \cdot k_e \cdot 2 \, \pi)} \qquad \text{(Eq. 8)}$$

In the following oscillogram we have measured the motor run-up. The first significant speed value (after the 3rd PWM cycle) is 44 mV. This corresponds to a speed of approx. 1.32 s^{-1}. But remember, that the bypass diodes cause an offset that has not been taken into account here!

In the first two PWM cycles, we do not obtain an evaluable BEMF voltage, but from the third cycle onwards the signal is clearly different from zero. Within the third cycle of 8 ms, the motor only moved by 0.0106 revolutions, which gives an impression of the positioning accuracy that can be achieved with this method.

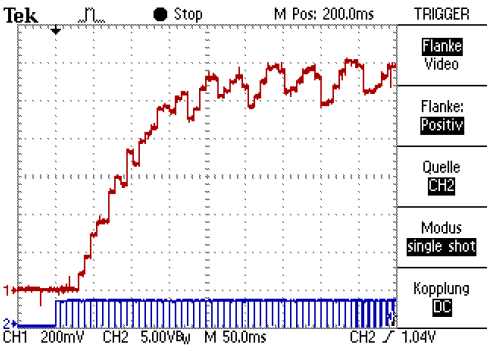


Figure 43: V_{BEMF} and PWM signal at motor run up

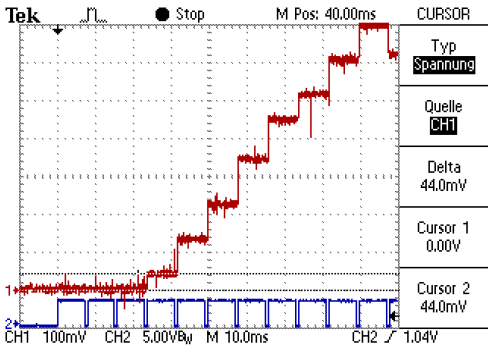


Figure 44: V_{BEMF} and PWM at Motor run up, Detail

5 Motor simulation with LTspice XVII

To gain an even deeper understanding of our circuit, we carried out an LTspice simulation. The following figure shows the circuit diagram.

5.1 LTspice Circuit diagram

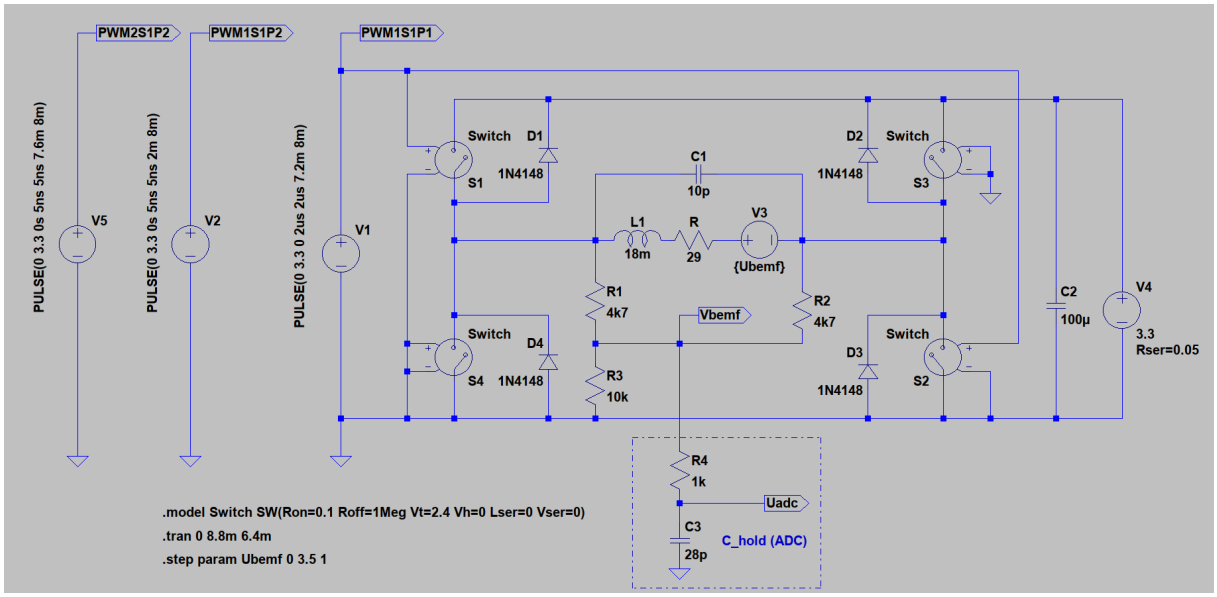


Figure 45: LTspice circuit diagram

Component V1 (**PWM1S1P1**) generates our PWM signal to control the H-bridge.
Component V2 (**PWM1S1P2**) and V5 (**PWM2S1P2**) are just simulated to plot our 3 PWM signals.

The H-bridge is built using 4 switches, which have been configured with **Ron = 0.1** ohms.

1N4148 type diodes are used to bypass the „FETs“.

Power supply **V4** has 3.3 volts with some series resistance (50 mΩ) and with a **100 µF** supporting capacitor.

Our voltage divider (**R1 to R3**) uses the values as used in the calculations before.

As the simulations showed, interference suppression capacitors (**C1**) above approx. 100 pF in parallel with the motor cause oscillations when measuring the back EMF, so they should only have very small capacitances or preferably be omitted altogether.

As $\mathbf{U_{BEMF}}$ is dependent on the speed, it was simulated using a STEP directive from 0 to 3.5 V in steps of 1 V.

6 Data Logger

Starting from version 0.8, the software logs up to 1024 readings of VBEMF and the motor current in two arrays of size LOGSIZE (= 1024 actually):

```
volatile uint8_t bemf_log[LOGSIZE];  ///< data logger: g_vbemf >> 4
volatile uint8_t curr_log[LOGSIZE];  ///< data logger: g_mAx10 >> 2
```

To reduce memory space, the 12 bit value `g_vbemf` is shifted 4 bit, and the motor current `g_mAx10` is shiftet 2 bit to the right, so that the values fit into a single unsigned byte.

The data logger is active on every MOVE or HOME travel until the buffer is filled. If the MOVE or HOME travel ends before `LOGSIZE` readings have been written, the buffer is filled up with zeros.

The buffer content can be read by sending the following GET command via a web browser on our LAN:

```
http://192.168.2.131/logdata?
```

After this command, the ESP sends a request to the PIC controller to transmit all the 1024 data sets in the form

```
index i, bemf_log[i],curr_log[LOGSIZE]\n
```

The ESP saves all the data records in file `/logdata.csv`, where it can be downloaded via the LittleFS manager (p.e. `http://192.168.2.131/fs.html`) and processed p.e. with Excel or Calc. The following diagram shows a sample log with the raw values converted to mA and Volts respective ($mA = 4 * curr_log[i]/10$ and $U_{BEMF}[V] = 16 * 2{,}048 * bemf_log[i]/4095/0{,}405$):

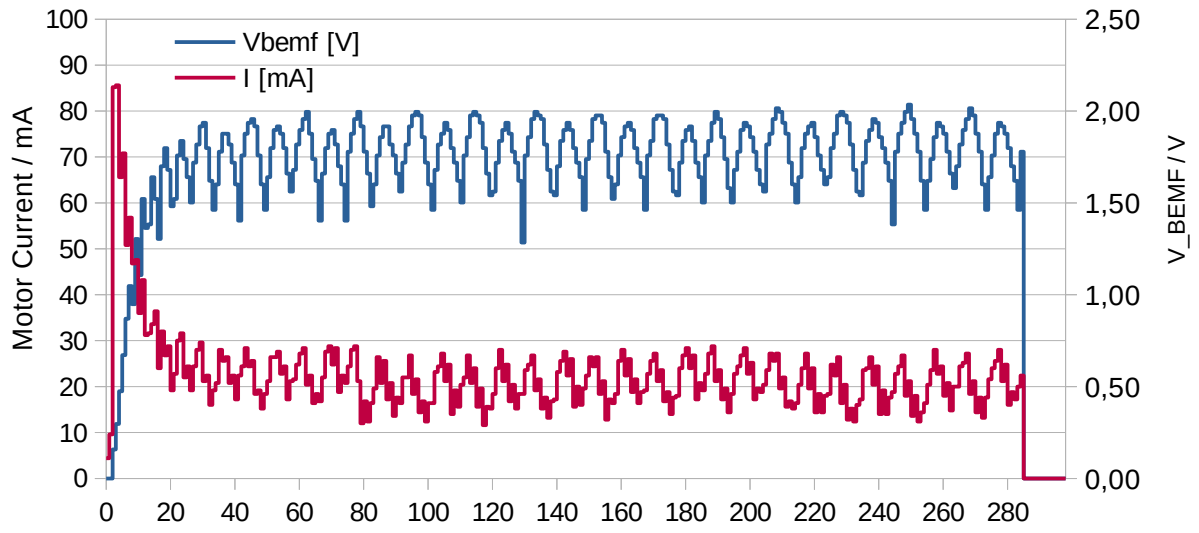


Figure 46: Sample log diagram (motor running on the desktop).
Notice: The right axis should be labeled U_BEMF, because the data has been scaled.

6.1 Measurements in the real field

The following logdata has been acquired in a real field environment with our floor heating valve. After a HOME travel with a current limit of 40 mA we moved the valve to 35 % with a current limit of 50 mA.

The second half of the V_{BEMF} signal shows the same low frequency oscillation which could be observed when running the motor without load. Obviously the motor is running faster than the valve can open, so the motor runs almost freely. The logfile contains 490 datasets.

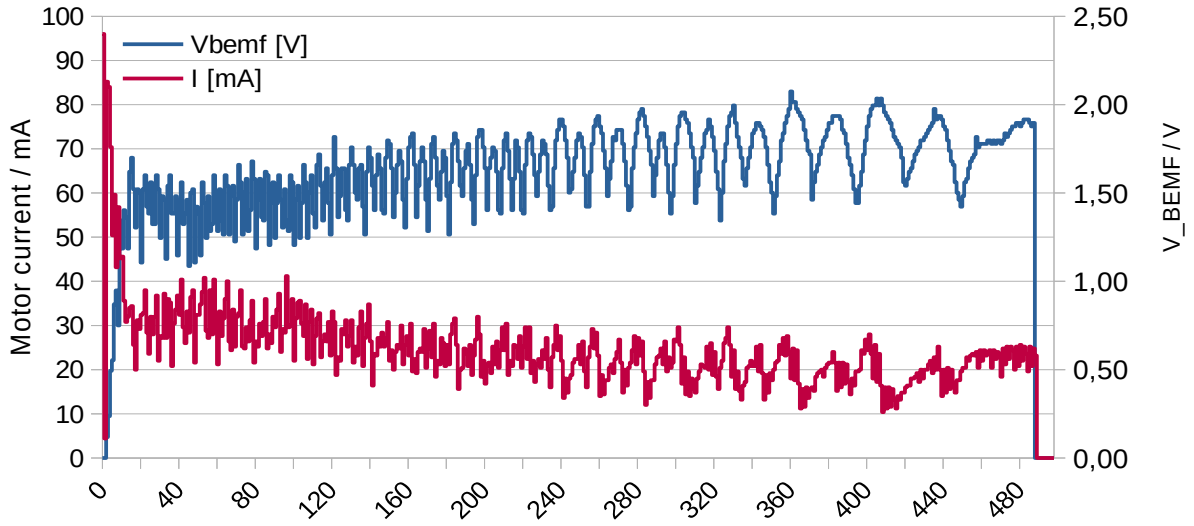


Figure 47: Measurement in the real field: Opening valve from 0 % to 35 %, Imax = 50 mA
Notice: The right axis should be labeled U_BEMF, because the data has been scaled.

Next, we moved the valve back to the closed position. After the startup we can see an almost linear increasing motor current, which corresponds to a decreasing V_{BEMF}, which is proportional to the motor speed. The motor stops after a fixed no. of time intervals. The logfile contains 489 datasets.

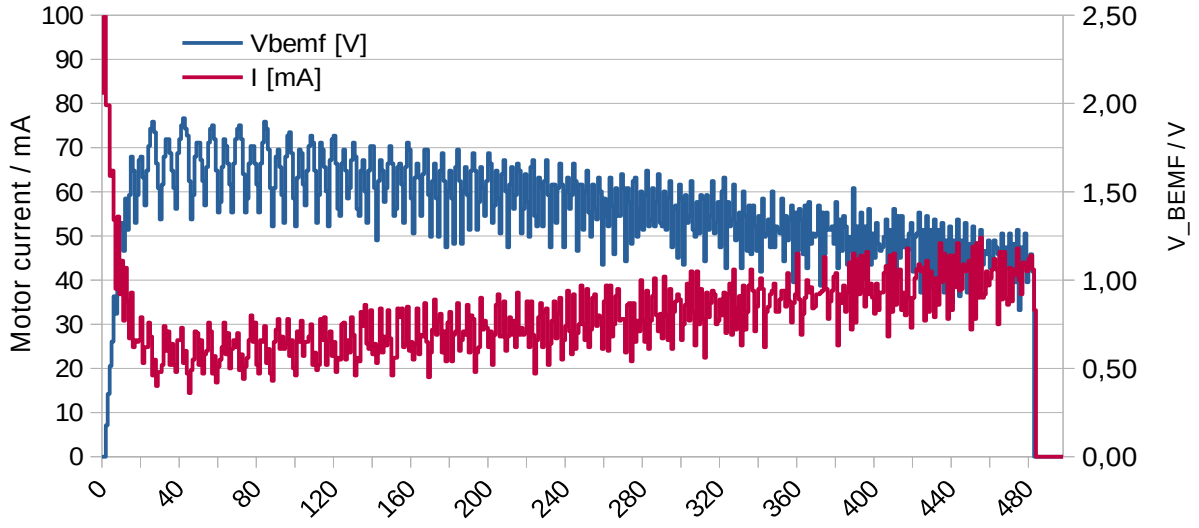


Figure 48: Measurement in the real field: Closing valve from 35 % to 0 %, Imax = 50 mA
Notice: The right axis should be labeled U_BEMF, because the data has been scaled.

7 Motor position control using BEMF?

In a first approach of the PIC software, a constant (experimentally determined) time interval of 100 ms for 1 percent travel was used for positioning. This worked surprisingly well on the lab bench, but in practice the motor speed was lower when closing than when opening, because less force is required when opening than when closing. Therefore, the drive must be referenced regularly (every few days) to maintain sufficient positioning accuracy.

But now we have all the information together to apply our findings in the firmware in order to achieve sufficient positioning accuracy. With analog measurement values, you always have measurement errors that can add up over time. There are people who are supposed to be able to filter out the motor revolutions from the BEMF, but with the signals available here and the minimal effort involved, I think this is out of the question.

Without a position sensor, we can only multiply the measured velocity *v(t)* by the very precisely measurable interval length *dt = 8 ms* to obtain the distance traveled *ds(t)*:

$$ds(t) = v(t) \cdot dt \qquad \text{(Eq. 9)}$$

First of all, we investigated the repeat accuracy we would achieve if we carried out a defined number of positioning processes (opening and closing the valve) with constant positioning intervals. As the motor speed depends on the load, this test had to be carried out with a varying load. We then measured the repeatability with a caliper gauge, which allowed us to determine the piston position with sufficient accuracy.The experiment was carried out on the laboratory bench. When opening, no load was applied; when closing, a random force was applied to the plunger with the thumb each time.

As a rough result, it was found that the plunger was offset by around **1 mm** from the starting position after 10 full open and close movements!

Next, we implemented the formula described above into the PIC firmware. We now sum up all raw readings of VBEMF into a (long int) variable VBsum, which gets reset when home position is set.

With a MOVE of 100 percent, VBsum reached a value of around 2 million, Each percentage therefore corresponds to a change in VBsum of around 20,000. So instead of moving at constant time intervals, we use VBsum as an estimation for the position. Each percentage value in the MOVE command setpoint must simply be multiplied (internally) by 20,000 and vice versa.

This approach was implemented in **PIC firmware v0.9** and then we repeated the experiment.

The results were disappointing. Although VBsum was approached within 0.5 percent in each case, the deviation of the piston after 10 passes was barely better than with the simple time control at approx. 1 mm.

So I decided to make both variants optional: If the constant `VBSUM_1pct` is initialized within the permissible value range in the EEPROM of the PIC-µC, the positioning will be estimated with VBsum, otherwise positioning is done with simple time control.

The constant `VBSUM_1pct` must be entered as 'unsigned short' in **EEPROM address @0x02/03** and in little endian (LB/HB) format.

This allows the piston stroke to be parameterized:

The `eewrite` function has been added to the Web-UI commands. As an example, to set `VBSUM_1pct` to a value of 25000, you can enter (adjust the IP address according to your settings):

```
http://192.168.2.131/eewrite?addr=2&data=25000
```

After a reboot of the PIC (power cycle), position control will be done by VBsum where a full 100 % travel will give VBsum approximately 100 * 25.000 = 2.500.000 ticks. In my case, the piston stroke with this setting was about 2.5 mm (instead of approx. 2 mm in the default control mode).

To return back to the time based position control, you have to write 0xFFFF (65535) into the EEPROM Adress 2/3 (and restart the PIC):

```
http://192.168.2.131/eewrite?addr=2&data=65535
```

Annotation: The limit values VBSUM_1pct_MIN and VBSUM_1pct_MAX are defined in main.h (currently 10000 to 40000). This means that time control is used if the EEPROM (0xFFFF) is deleted.

Outlook

The process will be investigated further when I find the time. It is probably related to the current measurement. Perhaps someone in the community has an idea of how the repeatability can be improved in this way?